Ders 4 Genişletilmiş Notu: Tasarım Çalışmaları, Kütüphane Örneği ve Birinci Derece Mantık

Ders 4 Genişletilmiş Notu: Tasarım Çalışmaları, Kütüphane Örneği ve Birinci Derece Mantık

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin planı
- 2. Tasarım nedir?
- 3. Tasarım çalışmaları (design studies)
- 4. Tipik bir tasarım çalışması raporu
- 5. Kütüphane örneği: gereksinimler
- 6. Kütüphane örneği: aday sınıflar ve eleme
- 7. Kütüphane örneği: özellikler, davranışlar ve ilişkiler
- 8. Kütüphane örneği: son dokunuşlar ve tasarıma geçiş
- 9. Biçimsel belirtim (formal specification)
- 10. Önerme mantığı
- 11. Birinci derece mantık (FOL)
- 12. Niceleyiciler ve kapsam
- 13. İfadeleri kurgulamak
- 14. Niceleyici sırası ve değilleme
- 15. Küme işlemleri ve OCL'ye geçiş
- Örnek Sorular

Ders iki bölümden oluşur. İlk bölümde tasarımın ve tasarım çalışmalarının ne olduğu anlatılır, sonra bir kütüphane örneği üzerinden gereksinimlerden kavramsal modele nasıl gidildiği yeniden gösterilir. Aradan sonraki ikinci bölümde biçimsel belirtime geçilir: önerme mantığı hatırlatılır ve birinci derece mantıkla (first-order logic, FOL) belirtim ifadelerinin nasıl kurulduğu örneklerle anlatılır. Slayt destesi BLM5126-05'tir (71 slayt) (ekrandan, 00:20:00).

1. Dersin planı

Hoca günün konularını şöyle sıralar: önce tasarımın ne olduğuna ve bir tasarım çalışmasının nasıl yapıldığına hızlıca bakılacak. Ardından bir hatırlatma egzersizi olarak gereksinimlerden yola çıkıp bir sınıf diyagramının, yani kavramsal modelin nasıl bulunduğu ve bunun tasarıma nasıl dönüştürüleceği konuşulacak. Sonra işin matematik tarafına geçilip biçimsel belirtim (formal specification) tanımlarına ve bunun için kullanılabilecek araçlara, dillere bakılacak. Hocanın ifadesiyle "elimizde iki silah" vardır: bugün birine (birinci derece mantık) bakılır, diğeri (OCL, UML tarafındaki karşılık) sonraki derse kalır.

2. Tasarım nedir?

Hoca önceki derslerde anlatılanı hatırlatır: bir mimariyi oluşturmak için önce sistemin, en azından yazılım altyapısının mimariye uygun hazırlanması gerekir ve bu süreç iki etaptan oluşur. İlk etap problemi anlama aşamasıdır; adına analiz denir. Çözümü oluşturma sürecinin

ikinci adımına ise tasarım denir. Buradaki tasarım, belirli bir mimariye uygun olacak şekilde düşünülerek oluşturulması gereken yazılım uzayı modelidir.

Tasarım yapılırken işlevsel olmayan gereksinimler göz önüne alınır: performans, kullanıcı sayısı, tepki süresi ve daha pek çokları. Hoca bunları en bilindik ve en basit örnekler olduğu için seçtiğini söyler. Bu gereksinimlere göre yazılım sınıfları, hatta dağıtım (deployment) planı ve bileşen yapıları farklı şekillerde tasarlanabilir. Analiz modeli yazılıma dönüştürülürken "kim kime nasıl servis vermeli, bunları nasıl tasarlamalıyım" gibi sorular düşünülüp çözülmek zorundadır.

Slayttaki tanım (ekrandan, 00:20:00):

- Tasarım, kararlar vermektir.
  - Genellikle işlevsel olmayan kriterlerden hangisinden ödün verileceği ile ilgilidir.
- Müşteri, son kullanıcı, teknik beklentiler, rakip ürünler bu kararları etkiler.
- Bazen de daha detaylı analiz gerekir: simülasyonlar, prototipler, tasarım çalışmaları.

Hoca, işlevsel olmayan gereksinimlerin hepsinin aynı anda karşılanamayacağını vurgular. Bazı şeyleri yaparken seçilen mimari ya da çözüm yüzünden pek çok şey yapılamayabilir; bu yüzden bazılarından ödün vermek gerekir. Verilen kararların toplamı tasarımı oluşturur.

Bazı durumlarda karar vermek için ek çalışma gerekir:

- **Simülasyon:** "Şöyle bir orta katman yapısı ve şu şekilde hizmet veren servisler düşünüyorum" denip bunun üzerinde küçük bir kavram kanıtı (proof of concept) çalışması yapılabilir. Yük testi yapar gibi bin ya da on bin kullanıcı geldiğinde belli bir zaman aralığında tepkilerin nasıl olduğu ölçülür ve tasarım kararları buna göre değiştirilebilir.
- **Prototip:** Birden fazla çözüm ya da küçük çözümler farklı farklı denenir. Bu daha fazla emek ister, çünkü birden fazla çözüm üretip işler hâle getirmek gerekir.
- **Tasarım çalışması:** Sonraki bölümün konusudur.

3. Tasarım çalışmaları (design studies)

Hoca tasarım çalışması kavramının yazılım dünyasında üretilmediğini, bina mimarisinden miras alındığını söyler. Bir mimar bina tasarlarken ilk önce bir tasarım çalışması yapar. Bir tasarım uzayı vardır: problemin üzerinde yaşadığı uzay, yani kavramsal model. Binalar somut, dokunulabilir yapılar olduğu için örneklemek kolaydır: mimarlar bu uzayı daha iyi hissetmek için ölçekli modeller yapar.

Hocanın gösterdiği örnekte eğimli bir arazinin üzerine bir yapı yapılmaya çalışılmaktadır. Farklı ölçekli modellerde "şurayı merdivenle mi yapsak, şu tarafı mı büyütsek, tek parça mı olsa, kot farkını duvar gibi mi düzeltsek" gibi farklı çözümler denenir. Bu, prototiplemeye benzer: aynı problem uzayında çalışacak farklı tasarım yaklaşımları küçük modeller hâlinde oluşturulur. Aynı yaklaşım yalnız binada değil, araba, uçak ve giysi tasarımında da kullanılır (kumaşı nasıl katlasak, yakası nasıl olsa gibi).

Sonuçta yapılan iş, tasarımı etkileyen faktörlerin titiz ve sistematik bir şekilde değerlendirilmesidir:

1. Tasarımın başarılı olduğunu hangi ölçütlerle ölçeceğiniz, yani kriterleriniz belirlenir.
2. Bu kriterlere göre ölçüm yapılır.
3. Hangi ölçüm sonuçlarının tatmin edici olduğu belirlenir.
4. Hangi yaklaşım (prototip, ölçekli model) daha uygunsa onunla yola devam edilir.

Hocanın bina örneğinde kriter yağmurdur: yapı Karadeniz kıyısında, çok yağmur alan bir yerdedir. Kot farkında su kalmaması ya da belli bir miktar su birikmesi bir kriter olabilir; ölçü birim zamanda tahliye edilebilen su miktarıdır. Ölçüm yapıldığında belki yokuş şeklindeki çözüm en iyi performansı verir, duvar şeklindeki çözüm ise suyun akmasını engeller ve sonunda duvar yıkılır. Kriterleri en iyi karşılayan tasarımla devam edilir.

Yazılımda da kavramsal model elinize alınır ve olası farklı çözümler geliştirilir: birinde bir mimari, diğerinde başka bir mimari denenir, ya da belirli kalıplar kullanılır ("olay tabanlı yapsak ne olur?", "farklı bir model kurgulasak ne olur?"). Böylece "bu kavramsal uzayda şu model daha iyi çalışacaktır" sonucu ortaya çıkarılır.

Hocanın vurgusu: En azından iki farklı şeyin denenmesi mutlaka tavsiye edilir. Bu aynı zamanda zaman kaybıdır: sonunda tek çözüm üretilecektir, ama ona ulaşana kadar iki farklı yoldan çözüm önerisi üretip belki prototiplerini çalıştırmak gerekir.

Binada göz önüne alınan nesnel faktörlerin yazılımdaki karşılıkları:

- **Maliyet:** Çözümün maliyeti çok yüksekse tasarım uygulanabilir (feasible) olmayabilir.
- **İnşaat malzemesinin bulunabilmesi:** Yeniden kullanılabilir (reusable) bileşenler ya da daha önce yapılmış parçalar var mı? Kullanılması düşünülen kütüphanelere erişim, lisans alınabilir mi? Yoksa her şey sıfırdan mı yazılacak?
- **Yasalar ve kısıtlar:** Lisans problemleri var mı, bunlarla yaşanabilir mi?
- **Trafik ve çevre:** Yazılım bir sunucuda çalışıyor ve uygulama sunucusunda başka hizmetler de varsa, yeni yazılım onların trafiğini etkileyecek mi, bant genişliğini daraltacak mı, sistem kaynaklarını yetersiz hâle getirecek mi?
- **Performans, bellek, üretim zamanı:** Bunlar özetle işlevsel olmayan gereksinim başlığı altındaki kıstaslardır.

4. Tipik bir tasarım çalışması raporu

Tasarım çalışmasında örnek çalışmalar yapılır, benzer çözümler üretilir ve sonuçlar raporlanır. Slayttaki rapor genellikle dokuz bölümden oluşur (ekrandan, 00:30:00):

1. **Bağlam:** Geçmiş, motivasyon, anahtar kelimeler ve tanımlar. Niye yapıldığı, ne yapıldığı, problemin ne olduğu ve varsa özel tanımlar.
2. **Araştırma soruları:** Bağımlı değişkenler cinsinden karşılaştırılan işlevsel olmayan gereksinimler ve etkilenen faktörler. Slayttaki örnek: ön işleme hesapları değişince zaman ve bellek kullanımları nasıl değişiyor? Hoca bunu "bir şeyi değiştirdiğimde kim nasıl etkileniyor?" sorularının açıkça ortaya konması, taranan uzayın önemli noktalarının yazılması diye açıklar.
3. **Konuların tanımları:** Programımız ve çözüm yaklaşımlarımız neler?
4. **Deneye özel koşul ve durumlar:** Kullanılan konfigürasyon özellikleri; tekrar edilebilirlik için gereklidir. Hocanın örneği: "Bunu bir Kubernetes ortamında, şu özelliklere sahip konteynerlerle çözdüm."
5. **Değişkenler (bağımlı ve bağımsız):** Değiştirilen durumlar bağımsız değişkenleri, bunlarla elde edilen sonuçlar bağımlı değişkenleri oluşturur. Durumların adı, tanımı ve ölçü birimi (ör. saniye) tanımlı olmalıdır. Hocanın örnekleri: şu modülü, şu bileşeni, şu ağ geçidini (gateway) ya da o ağ geçidindeki şu kuyruk mekanizmasını denedim.
6. **Metot:** Denemeler, ölçü alet ve araçları, çalışan konular ve kullanılan argümanlar (ör. grid boyutu).
7. **Deney sonuçları:** Elde edilen sonuçlar.
8. **Tartışma:** Sonuçların yorumlanması ve değerlendirilmesi; anormal sonuçlar, neler yapılabileceği. Hocanın örneği: "Şunun performansa etkisi şu kadar artı, ama bellek kullanımını şu kadar artırıyor; bu da şöyle bir altyapı maliyeti getiriyor."
9. **Sonuç:** Araştırma sorularına açık cevaplar verilebilmelidir: işlevsel olmayan gereksinimlerin ne kadarı nasıl karşılanıyor ve bu sistemde neyi etkiliyor?

Hoca bunun uzun, yorucu ve sıkıcı bir iş olduğunu kabul eder. Ancak özellikle resmî ya da kontrata dayalı, kritik uygulamalarda (ör. bir askerî projede ya da roket fırlatmayı kontrol eden bir yazılım ekibinde) mutlaka yapılması gerekir. Takvimin sıkışık olduğu işlerde bu ayrıntıda yapılması beklenmez ve her zaman yapmak zorunlu değildir. Ama ciddi bir iş yapılıyorsa ve gerçekten en iyi, optimize çözüm aranıyorsa, onu bulmanın yolu bu şekilde denemeler yapmaktan geçer.

5. Kütüphane örneği: gereksinimler

Hoca, "kavram nasıl anlaşılır, sonra tasarlanmak üzere nasıl modellenir" sorusunu hatırlatmak için çok basit bir kütüphane örneği çözer. Hikâyede iki arkadaş vardır. Biri küçük bir kütüphane işletmektedir ve kayıtları elle ya da Excel ile tutar: kim kaç kitap aldı, kaç kitap alabilir, ödünç verilenler, cezalar. Bazı şeyleri karıştırdığından ya da takip edemediğinden yakınır ("bu şunu ödemiş miydi, öğrenci olduğu için şu kadar limiti var mıydı?"). Diğer arkadaşı ona bir bilgi sistemi geliştirmeyi önerir ve "kütüphanen nasıl çalışıyor, anlat" der. Hoca bunun gerçek

hayattaki karşılığını şöyle verir: müşteriyle ya da iş birimiyle konuşulur ve problem uzayının ne olduğu, onu oluşturan parçaların neler olduğu bulunmaya çalışılır.

Konuşmadan çıkan bilgiler slaytta gereksinimler listesine dökülmüştür (ekrandan, 00:40:00):

1. Her ziyaretçi sistemde olduğu sürece bir tekil kütüphane kartına sahiptir.
2. Kütüphane her ziyaretçi için en az ad, adres, telefon, kütüphane kartı numarasını bilmelidir.
3. Kütüphane herhangi bir anda bir ziyaretçinin ödünç aldığı parçaları, teslim zamanlarını, ödemediği gecikme cezalarını bilebilmeli veya hesaplayabilmelidir.
4. 12 yaş altındaki çocuklar için özel bir kısıtlama vardır. Bir kerede en fazla 5 parça ödünç alabilirler.
5. Ziyaretçi kitap veya DVD ödünç alabilir.
6. Kitaplar üç hafta için ödünç alınabilir. Çok satan kitaplar için limit iki haftadır.
7. DVD'ler iki hafta için ödünç alınabilir.
8. Gecikme cezası her parça için günlük on TL'dir. Gecikme cezası parçanın fiyatından daha yüksek olamaz.
9. Kütüphanede ödünç alınamayan başvuru kitapları ve dergiler de vardır.
10. Ziyaretçi, kütüphanede olmayan bir kitabı ya da DVD'yi talep edebilir.
11. Ziyaretçi bir parça için sadece bir kez süre uzatma yapabilir. Parça için çok talep varsa, parçayı geri getirmelidir.

Hocanın anlatımından bazı ayrıntılar:

- Kütüphane kartı, o yıllarda moda olan sadakat (loyalty) kartları gibi düşünülmüştür. Üzerinde kredi kartı gibi bir numara vardır; ziyaretçi bu numarayla tekil olarak tanımlanır.
- Kütüphanede "müşteri" demek garip kaçtığı için "ziyaretçi" denir.
- Çocuklara sınır konmasının nedeni, aldıklarını getirmeme, kaybetme ve cezaları ödememe durumlarının sık yaşanmasıdır. Beş parça sayısı düşmeden, yani bir şey getirilmeden beşten fazla verilmez.
- Çok satan (bestseller) kitaplar çok talep gördüğü için süre üç haftadan iki haftaya düşürülmüştür. DVD'ler her hâlükârda iki haftalığına verilir.
- Talep etme, ilçedeki ya da diğer ilçelerdeki kütüphanelerle haberleşerek çalışır. Bazı kütüphanelerin bilgi sistemi vardır ve sorulduğunda "şu var, şu yok, elimde üç tane var" gibi cevap verir. Kütüphane bir tür kütüphaneler arası oluşum gibi, başka kütüphanelerdeki parçaları sorgulayıp ziyaretçisine verebilir.
- Süre uzatma: ders kitabı ya da uzun bir roman üç haftada bitmeyebilir; kimse istememişse bir kez daha verilir. Parça için talep varsa, özellikle belli bir sınırın üzerindeyse (hocanın örneğinde 5'ten fazla kişi bekliyorsa) uzatma yapılmaz ve parça bekleme kuyruğundaki sıradaki ziyaretçiye verilmek üzere geri getirilmelidir.

5.1 Gecikme cezası örneği

Hoca 8. gereksinimin ikinci cümlesinin neden gerektiğini sayısal bir örnekle anlatır. Kitap üç haftalığına alınmış, süre dolduktan sonra geç getirilmiştir; ceza günlük 10 liradır.

- Bir hafta geç getirilirse: $7 \times 10 = 70$ lira.
- Üç hafta geç getirilirse: $21 \times 10 = 210$ lira.
- Kitabın fiyatı 50 lira ise 210 liralık ceza saçma olur: kişi o parayla kitabın iki tanesini alıp getirebilir.

Bu yüzden ceza en fazla ürünün fiyatı kadar (ya da hocanın sözlü eklediği gibi fiyatın belli bir çarpanı kadar) olabilir. Slayttaki kurala göre bu örnekte kesilecek ceza $\min(210, 50) = 50$ liradır (ara adım eklendi).

5.2 Gereksinimlerden ne çıkıyor?

Hoca kütüphanenin işin yalnızca "maskesi" olduğunu söyler. Bazı kavramların tanımlanması, bazılarının ilişkilendirilmesi, sonra bir genelleştirme yapılması gerekir. Parçalar (kitaplar, DVD'ler) kimi ödünç verilebilen, kimi verilemeyen nesnelerdir; demek ki farklı davranış ya da kategorilere ait olmalıdır. Sistemin hizmet olarak sunması gereken işlevler de vardır: teslim zamanlarını sorgulamak, cezayı ziyaretçi bazında hesaplamak, bir talep geldiğinde başka sistemlerle konuşarak "şu sende var mı, kaç tane var?" diye sormak. Bu son işlem için başka

sistemlerle konuşmayı sağlayan bir servis gerekir. Kısacası modellenen şey, kavramları alan nesneleri (domain object) olarak tutan, çeşitli hizmetler veren ve gerektiğinde başka sistemlerle bilgi alışverişi yapan bir bilgi sistemidir. Hoca örneğin çok basitleştirilmiş olduğunu, tek kişinin işlettiği böyle bir kütüphanenin gerçekte olmayacağını da ekler.

6. Kütüphane örneği: aday sınıflar ve eleme

Önceki derste anlatılan yöntem uygulanır: gereksinim metnindeki isimler işaretlenir. Slaytta adaylar kırmızıyla, özellik olabilecek isimler ve onları niteleyen sözcükler yeşille işaretlenmiştir (ekrandan, 00:50:00). Hoca "sistem" sözcüğünü de hatırlatır: istenirse bir sınıf gibi modellenebilir, ama genellikle kavramsal uzayda ayrıca yer verilmez, çünkü uzayın kendisi zaten sistemi oluşturur.

Adayların değerlendirilmesi:

- **Adres, telefon:** Ayrı kavram olarak modellenmeyecek; başka bir kavramı, yani ziyaretçiyi tanımlayan öz niteliklerdir.
- **Teslim zamanı:** Parçalarla ilgilidir.
- **Gecikme cezası:** İlk tasarımda parçanın içine konabilir. Ama cezanın farklı davranışları olabilir (gün sayısı çarpı 10 ya da parçanın fiyatını geçemez), bu yüzden şimdiden ayrı tutulabilir.
- **Çocuk ve 12 yaş:** Çocuklar için bir kısıtlama vardır. Bugün 5 parça olan sınır yarın 10 ya da 3 olabilir, belki başka bir kriter gelir; bu yüzden kısıtlamalar da ayrı tutulabilir.
- **Kitap, DVD:** Kendilerine göre farklı özellikleri olan parçalardır.
- **Limit (3 hafta, 2 hafta):** Bir özelliktir.
- **Çok satan kitap:** Kitaptan ayrı bir şey gibi görünür, o da modellenebilir.
- **10 TL:** TL bir hizmet sınıfı ya da sayım (enumeration) sınıfı gibi düşünülebilir. Bugün TL olur, yarın dolar ya da euro olabilir; değişime açık bir nokta olduğu için şimdiden kategorik ya da parametrik yapmakta fayda vardır.
- **Fiyat:** Parçanın özelliğidir.
- **Başvuru kitapları ve dergiler:** Aday sınıflardır.

İlk sınıflar slayttaki el çizimidir (ekrandan, 00:44:10):

```
KütüphaneKartı { KKartNo }        Ziyaretçi { ad, adres, telefon }
Ceza { }                          Parça { fiyat }        Çocuk { yaş }
BaşvuruKitabı { }   Dergi { }   Kitap { }   DVD { }   ÇokSatan { }
Para { }   Tarih { }
```

Para ve tarih hizmet sınıflarıdır.

> **Hocanın vurgusu:** Bazı adayların gereksiz olduğu baştan bilinse de, tanınmayan ya da iyi hâkim olunmayan bir uzayda önce bütün olası adayları ortaya koymak, hiçbirini atlamamak, sonra gerekiyorsa elemek ya da değiştirmek daha doğrudur. Bu, bir şeyi kaçırma olasılığını azaltır. Eleme düşünerek, konuşarak ve tartışarak yapılır; ekip hâlinde yapılırsa sonuç daha başarılı olur.

6.1 Kütüphane kartını elemek

Kütüphane kartı ayrı bir kavram olarak modellendi, ama yalnız kart numarası tutuyor ve ziyaretçiyle birebir ilişkide: her ziyaretçinin bir kütüphane kartı var, başka da yok. O zaman ayrı bir sınıf olarak modellemek gerekli değildir. Kart numarası Ziyaretçi'nin içinde benzersiz bir özellik olursa yeter.

6.2 Çocuğu elemek

Çocuk aslında bildiğimiz bir ziyaretçidir; ziyaretçiden tek farkı yaşıdır. Ziyaretçilerin de yaşı olmak zorundadır ve sistem yaşa göre farklı davranışlar sergileyebilir. Hoca alt sınıf yaklaşımının sorununu şöyle anlatır: çocuk 12 yaşına kadar ayrı bir sınıftadır, sonra 13 yaşına girer. Çocuk sınıfından türetilmiş bir nesne, Ziyaretçi sınıfından türemiş bir nesneye nasıl dönüştürülecek? Nesneyi öldürüp değişkenlerini kopyalayıp yeni bir sınıftan nesne yaratarak değerleri oraya mı aktaracaksınız? Bu saçma olur.

> **Hocanın vurgusu:** Esas sınıflarımız değişmez, tutarlı yapılar olmalıdır. Çocuk kavramına ihtiyaç yoktur; o bir ziyaretçidir ve onu tanımlayan yaş öz niteliği Ziyaretçi'nin içine konur.

6.3 Çok satanı elemek

Aynı mantık çok satan kitaba da uygulanır. Kitap çok satan sınıfından türetilmişse, rağbetten düşüp normal kitap hâline geldiğinde ne yapılacak? Çok satan olmak kitabın bir özelliğidir: Kitap'ın içine "çok satan mı, değil mi" diye bir özellik konur. Başvuru kitapları ve dergilerin ödünç alınamadığı zaten bilinmektedir; ceza, para ve tarih ise henüz yerinde durur.

7. Kütüphane örneği: özellikler, davranışlar ve ilişkiler

7.1 Özellikler

Ziyaretçinin o ana kadar görülen özellikleri eklenmiştir. Parça tarafında:

- **Fiyat:** Ceza fiyatla karşılaştırılmalıdır, çünkü fiyattan fazla ceza kesilemez.
- **Teslim tarihi:** Ödünç verildiğinde her parçanın kendine göre bir teslim tarihi olmalıdır.
- **Uzatma sayısı:** Bir kez uzatılabildiği için, ilk mi ikinci mi ödünç verildiği takip edilmelidir.
- **Ödünç mü:** Parçanın şu anda ödünç verilmiş olup olmadığı (true ya da false).
- **Ödünç verilebilir:** Başvuru kitapları ve dergilerde false, kitap ve DVD'lerde true olur. Bu kontrol edilerek ödünç verme davranışı sergilenir ya da sergilenmez. Hoca bunu bir yol olarak görür ve şimdilik fena görünmediğini söyler.

7.2 Davranışlar

Davranışlar fiillerden gelir (ödünç almak, gecikme cezasını ödememek, talep etmek). Slayttaki el çiziminde Ziyaretçi sınıfı şöyledir (ekrandan, 00:50:00):

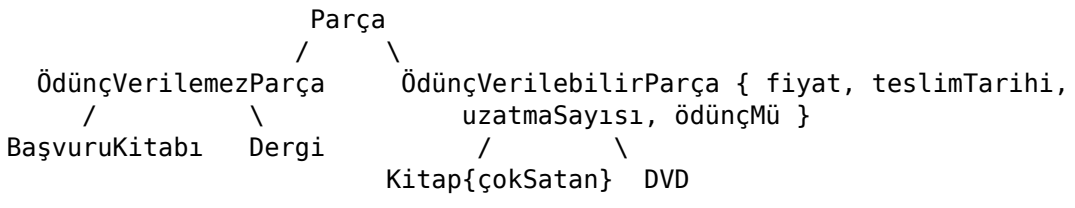
```
Ziyaretçi
  ad, adres, telefon, KKartNo, yaş
  ödünçAldıkları
  teslimTarihleri
  ödenmemişCezalar
  ödünçAl
  talepEt
```

Hocanın gerekçeleri: bir ziyaretçinin neleri ödünç aldığı sorgulanabilmelidir; teslim tarihleri sorgulanabilmelidir ve bunu ziyaretçi sorgulayacağı için davranış onun içine konur; ödemediği cezalar bilinebilmelidir; ödünç alabilir; ve beşinci davranış olarak talep edebilir. Talep etme, kütüphanede olmayan bir parça için diğer sistemlere sorgu yapılmasını tetikleyen davranıştır. Metotların içi şu anda boştur.

7.3 Genelleştirme

Parçanın içindeki "ödünç verilebilir mi" özelliği aslında farklı kavramlar hâlinde modellenirse daha doğru olur. Parçaların altında ödünç verilemez parçalar ve ödünç verilebilir parçalar olur. Başvuru kitabı ve dergi ödünç verilemez parçadan, kitap ve DVD ödünç verilebilir parçadan türer. Ziyaretçinin ödünç alma ve talep etme ilişkileri yalnız ödünç verilebilir parça sınıfına bağlanırsa, ziyaretçinin dergi ve başvuru kitaplarını ne ödünç alabileceği ne de talep edebileceği otomatik olarak modellenmiş olur.

Slayttaki diyagram (ekrandan, 00:51:30):



```
Ziyaretçi --ödünçAlır--> ÖdünçVerilebilirParça
Ziyaretçi --talepEder--> ÖdünçVerilebilirParça
(Ceza, Para, Tarih ayrı kutular olarak duruyor)
```

Slayttaki notlar: Ziyaretçi ile Parça arasında «ödünçAlma» ve «talepEtme» ilişkileri vardır. BaşvuruKitabı, Dergi, Kitap, DVD hepsi özel birer parçadır; bunlar hiyerarşik genelleştirilip «ödünçVerilebilir» özelliği kaldırılabilir. Dikkat: bu soyutlama ile «ödünçAlma» ve «talepEtme» ilişkileri artık «ödünçVerilebilir» sınıfına olmalıdır.

7.4 Talep etmeyi yeniden düşünmek: Başlık sınıfı

Hoca bu mantığın ilk başta doğru göründüğünü, ama iyileştirilmesi gerektiğini söyler. Bir ziyaretçi başvuru kitabını ödünç almak istemeyebilir ama kütüphanede okumak isteyebilir: “Sende yok ama başkasında var, getirtmeni istiyorum.” O zaman talep edebilmelidir. Talebi Parça’ya bağlamak akla gelir; böylece hepsi talep edilebilir ama yalnız ödünç verilebilir olanlar ödünç alınabilir.

Ancak talep doğrudan Parça’ya bağlanırsa başka bir sorun çıkar. Kütüphanede bir kitabın beş kopyası varsa, talep etme işlemi beşini birden talep etmek gibi olur. Çoğullama koyup tek bir nesneyi talep etmek de çözüm değildir: kullanıcı “senin beş kopyan var, ben özellikle ikinci kopyayı istiyorum, ötekilerin cildi kırışmış” diyemez. Kütüphanede beş kopyanın herhangi biri ödünç verilir ya da talebe karşılık döndürülür.

Çözüm başlıktır (title). Başvuru kitaplarının, dergilerin, kitapların ve DVD’lerin hepsinin bir başlığı vardır (ör. Don Kişot). Talep Başlık sınıfına bağlanır; Başlık, parçaları birleştirir (aggregation). Ödünç alma ise ödünç verilebilir parçada kalır. Böylece talep belirli bir başlığa yapılır (“ben Don Kişot istiyorum”), “üçüncü Don Kişot’u istiyorum” gibi bir şey söz konusu olmaz ve parçalar başlıklarıyla soyutlanmış olur.

7.5 Ödünç ilişki sınıfı

Ödünç alma ilişkisinin okuma yönü “Ziyaretçi ödünç alır ödünç verilebilir parçaları” şeklindedir; bu ilişki ödünçAl metodundan gelir. Hoca parçadaki özellikleri sorgular: teslim tarihi bir kitabı tanımlayan bir özellik midir? Uzatma sayısını takip etmek kitabın ya da DVD’nin sorumluluğu mudur? “Ödünç verildim mi” diyebilmesi gerekir mi? Önceki derste anlatılan ilişki sınıfları burada işe yarar: bunlar ne ziyaretçinin ne de parçanın özelliğidir, ödünç alma ilişkisinin kendi özellikleridir.

Slayttaki gerekçe (ekrandan, 00:57:10):

- uzatmaSayısı ve ödünçMü özellikleri aslında parçanın değil, ilişkinin özellikleri. Bir kitabı hayatı boyunca bir kez uzatmayız.
- «ödünç» isimli bir ilişki sınıfı yaratıp özellikleri ona atarız.
- Ceza da ödünç’ün bir özelliği olur.
- Ziyaretçi kitabı geri getirecek, ya kitabı bırakır da cezayı ödemezse??

Ödünç sınıfına teslim tarihi, uzatma sayısı, ceza ve getirme tarihi konur. Fiyat ise gerçekten kitabın ya da DVD’nin bir özelliğidir ve parçada kalır. Cezanın ödünç içine konmasının nedeni slayttaki son sorudur: ceza ayrı bir nesne olarak kalırsa, ziyaretçi kitabı bırakıp cezayı ödemediğinde ceza nesnesi ayrı yaşamaya devam eder. Ceza ödünç alma ilişkisinin içinde modellenirse bu durum ortadan kalkar.

8. Kütüphane örneği: son dokunuşlar ve tasarıma geçiş

Model artık yeterlidir, ama “biraz daha eli yüzü düzgün” hâle getirilebilir. Slayttaki başlıklar: yaş ve ceza hesaplanan (türetilmiş) özelliklerdir; özelliklerin tipleri, çoğullamalar ve stereotipler eklenir. Son model, slayttaki el çiziminden metne dökülmüştür (ekrandan, 01:00:00):

```
Başlık                          Ziyaretçi
  adı: String                     ad: String
  yazar: String                   adres: String
  barkod: String                  telefon: String
                                  KKartNo: String
```

```
                                  doğumTarihi: Tarih
                                  /yaş: Integer
                                  ...
Başlık "*" <--talepEder-- "*" Ziyaretçi
Başlık 1 <>-- * Parça                   (birleşme)

Parça
  <- ÖdünçVerilemezParça  <- BaşvuruKitabı, Dergi
  <- ÖdünçVerilebilirParça
       günBaşınaCeza: Para = 10
       ödünçPeriyodu: Tarih
       fiyat: Para
       <- Kitap { çokSatan: Boolean }
       <- DVD

ÖdünçVerilebilirParça ---- Ziyaretçi    (ödünç alma ilişkisi)
        :
      Ödünç (ilişki sınıfı, kesikli çizgiyle bağlı)
        uzatıldıMı: Boolean = false
        teslimTarihi: Tarih
        /ceza: Para
        neZamanGeldi: Tarih

<<utility>> Para    <<utility>> Tarih
<<utility>> İşletimSistemi { tarihVer(): Tarih }
```

Hocanın açıklamaları:

- Başlık ile Ziyaretçi arasında çoka çok ilişki vardır: bir ziyaretçi n tane başlık talep edebilir, bir başlık n ziyaretçi tarafından talep edilebilir.
- Para ve Tarih `<<utility>>` stereotipiyle verilir. Tarih için işletim sisteminin tarih veren bir davranışı olabilir: “saat kaç, tarih ne?” diye sorulabilir.
- Gün başına ceza Para tipindedir ve varsayılan değeri 10’dur (günde 10 lira). Ödünç verme periyodu Tarih, fiyat Para tipindedir.
- Başlık ile Parça arasında bire çok ilişki vardır: bir başlık altında n parça olabilir (ör. Don Kişot’tan 8 kopya).
- Barkod: başlıkları benzersiz tanımlamak için kütüphanenin barkodları vardır. Barkod olarak ISBN kullanılırsa kavram karmaşası yaşanabilir, çünkü farklı yayınevlerinden çıkan aynı kitabın ISBN’leri farklıdır. Bu modelde barkod kütüphanenin kendi tanımladığı barkoddur; kütüphane kendini yayınevinden soyutlamıştır ve “illa şu yayınevinin Don Kişot’unu istiyorum” denemez. Eğer bu gerekseydi model çalışmazdı: yayınevini ya da kimlik numarasını tutan ayrı bir kavram bulunur, başlık ve yayınevi birlikte bağlanır ve talep belki ISBN’e yapılırdı. Örnek sade kalsın diye bu ayrıntı düşünülmemiştir.

Davranışlar yer kaplamasın diye son diyagrama yazılmamıştır. Bundan sonra yapılması gereken, bu davranışların nasıl çalışacağının modellenmesidir. Hoca tasarıma geçerken sorulacak soruları sıralar:

- Bu kavramlar hangi yazılım sınıflarına dönüşecek? Genelleştirme yapılan parçalar gerçekte nerede duracak: veri tabanında mı, başka bir sistemde mi, bir veri depolama sisteminde mi?
- Ödünç verme hizmetleri çok yoğunsa farklı servislere bölmek gerekecek mi? Bunlara erişim için bir ön yüz tanımlanacak ve ziyaretçiler ödünç servisinden bu ön yüz üzerinden mi yararlanacak?
- Ziyaretçilerin işlem sırasına, kuyruğuna nasıl girmesi gerekecek; mesaj kuyrukları nasıl oluşturulacak?

Bu tasarım soruları alternatifleriyle ortaya konur, tasarım çalışmaları yapılır, çalıştırılıp raporlanır. Örneğin “1 milyon kullanıcıya 10 saniyenin altında hizmet sunabilme” gibi bir işlevsel olmayan gereksinimi karşılamak için hangi prototipin ya da önerilen tasarımın daha doğru olduğu gösterilerek gerçek yazılım tasarımına geçilir. Mimariyi düşünmek de aslında işlevsel olmayan gereksinimlerden hangisinin ne kadar karşılanacağını tartıp bulmaktır.

9. Biçimsel belirtim (formal specification)

Aradan sonra hoca biçimsel belirtime geçer. Tasarımda yapılmaya çalışılan şey, programın, yani yazılan kodun ne iş yapacağını ortaya çıkarmak ve anlatmaktır. Programın yaptığı şeyi tanımlamaya belirtim (specification) denir.

Belirtimler farklı kaynaklardan ve biçimlerde gelebilir:

- Müşteriden, son kullanıcıdan ya da analiz takımından gelebilir.
- Gayriresmî olabilir: Jira ya da TFS gibi bir iş takip aracına girilmiş iki satırlık bir talep gibi.
- Yapılandırılmış bir belge olabilir: kontratlı bir işte, ör. savunma sanayii için geliştirilen bir projede her şey kesin tanımlıdır ("şu işi yapacaktır, bu tepkiyi şöyle verecektir, raporunu böyle oluşturacaktır").
- Biçimsel, matematiksel ifadelerle tanımlanabilir.

Bu ders kapsamında bakılan, son seçenektir: matematik notasyonu kullanarak değişmez, yani okuyan herkesin aynı şeyi anlayacağı kesinlikte ifadeler oluşturmak. Bunun için iki araç vardır:

- **Birinci derece mantık (first-order logic):** Matematik ve mantık dili kullanır. Bu dersin konusudur.
- **Nesne kısıt dili (Object Constraint Language, OCL):** FOL'un çok benzeri, ama daha "insancıl", okunaklı versiyonudur; programlamaya yakın, sözde kod gibi bir dil kullanır.

İkisiyle oluşturulan ifadeler de kesin ve değişmezdir ve birbirine dönüştürülebilir.

Hoca kullanılacak mantığı kategorize eder. Tasarım için tanımlar, akıl yürütmeler ve ispatlar yapılır; bunların resmî bir dille gösterilmesi gerekir. İlk yol önerme mantığıdır (propositional logic): liseden hatırlanan p'ler ve q'lar, doğruluk tabloları ve mantıksal denklikler. İkincisi bunun ilerisine geçen birinci derece mantıktır. Hoca Türkçede "ilk sıra mantığı" dediğini, farklı çevirilerin de olduğunu söyler (slayt başlığı "Birinci Derece Mantık"tır). Birinci derece mantık, çoklu nesnelerin özellikleri hakkında akıl yürütmeyi ve bunları resmî biçimde ifade etmeyi sağlayan bir mantık dilidir.

10. Önerme mantığı

10.1 Önerme nedir?

Önerme, ya doğru ya yanlış olan bir ifadedir. Hocanın örnekleri:

- "Köpek yavruları kedi yavrularından daha sevimlidir." Bir önermedir; bana göre doğru, size göre yanlış olabilir. Tersi ("kedi yavruları daha sevimlidir") de bir önermedir. Bu ikisi karşılıklı dışlayan (mutually exclusive) ifadelerdir: belirli bir anda yalnız biri doğrudur, diğeri kesin yanlıştır.
- "Bu listedeki son maddedir." Slayttaki üç maddelik listenin gerçekten son maddesi olduğu için doğru bir önermedir.

Önerme olmayanlar:

- Komutlar: "Kalemi eline al." Bir buyruktur.
- Sorular: "Bu akşam hava nasıl olacak?" Doğru ya da yanlış değildir.
- Anlamsız, belirsiz ifadeler.

İfadelerde önerme değişkenleri (variable) ve onları birleştiren bağlaçlar (connective) vardır. Değişkenler genellikle küçük harfle yazılır ve her biri bir önerme gibi doğru ya da yanlış olmalıdır.

10.2 Bağlaçlar

Slayttaki bağlaçlar (ekrandan, 01:30:00):

- **Mantıksal değil (NOT), $\neg p$:** $\neg p$ ancak ve ancak p yanlışsa doğrudur. Tersini alma (negation).
- **Mantıksal ve (AND), $p \wedge q$:** p ve q ikisi birden doğruysa doğrudur (conjunction). Herhangi biri ya da ikisi yanlışsa sonuç yanlıştır; bildiğimiz VE kapısıdır.

- **Mantıksal veya (OR),** $p \vee q$**:** En az biri doğruysa doğrudur; kapsayan veya (inclusive OR, disjunction).
- **Çıkarım (implication),** $p \to q$**:** "p implies q" diye okunur: p doğruysa q da doğrudur. Denk biçimi $\neg(p \wedge \neg q)$'dur; doğruluk tablosu ya da sadeleştirme yaparken bu biçim kullanılabilir.
- **İki koşullu (biconditional),** $p \leftrightarrow q$**:** "p if and only if q" diye okunur. $p \to q$ ve $q \to p$ demektir: her iki yönde de doğru olmalıdır.
- **Doğru (True),** $\top$**:** Normal T harfi.
- **Yanlış (False),** $\bot$**:** Ters çevrilmiş T harfi. Bir değişken yerine doğrudan doğru ya da yanlış bir ifade kullanmak gerektiğinde kullanılır.

10.3 Doğruluk tablosu

Hoca slaytta öbür işaretleri yazamadığı için ön ek (prefix) bir gösterim kullanır: `(! P)` değil P, `(| P Q)` P veya Q, `(& P Q)` P ve Q, `(=> P Q)` "if P then Q" ya da P implies Q, `(<=> P Q)` P if and only if Q, yani P, Q'ya denktir. Slayttaki tablo (ekrandan, 01:32:00):

P	Q	P ∨ Q	P ∧ Q	¬P	P → Q	P ↔ Q
T	T	T	T	F	T	T
T	F	T	F	F	F	F
F	T	T	F	T	T	F
F	F	F	F	T	T	T

Tablo python ile doğrulanmıştır. İki koşullu bağlaç, iki değer aynıyken doğru, farklıyken yanlıştır.

10.4 Operatör öncelikleri

Slayttaki kurallar (ekrandan, 01:33:20):

- Öncelik sırası: $\neg$, $\wedge$, $\vee$, $\to$, $\leftrightarrow$.
- $\neg$ takip eden değişkene yapışır; tek bir atomik parça gibi düşünülebilir.
- $\wedge$ ve $\vee$, $\to$'dan daha sıkı bağlıdır.

Slayttaki örnek:

$$\neg x \to y \vee z \to x \vee y \wedge z$$

parantezlendiğinde

$$(\neg x) \to ((y \vee z) \to (x \vee (y \wedge z)))$$

olur. $\neg$ x'e yapışır, $\wedge$ önce y ile z'yi bağlar, sonra $\vee$ gelir ve en son çıkarımlar uygulanır.

> **Ek bilgi (derste anlatılmadı):** Slayttaki parantezleme, ardışık $\to$ bağlaçlarının sağdan birleştiğini gösterir: $a \to b \to c$, $a \to (b \to c)$ diye okunur. Bu, mantık kitaplarında yaygın bir gelenektir; şüpheli durumda parantez kullanmak en güvenli yoldur.

10.5 Doğal dilden önerme mantığına

Hoca doğal dildeki bir cümlenin değişkenlere ve bağlaçlara nasıl dökülebileceğini gösterir.

Örnek 1. Değişkenler:

- A: "Bu akşam uyanık olacağım."
- B: "Bu akşam ay tutulmasını göreceğim."

Cümle: "Eğer bu akşam uyanık olmazsam ay tutulmasını göremem."

- "Uyanık olmazsam" A'nın tersidir: $\neg A$.

- “Göremem” B’nin tersidir: $\neg B$.
- “Eğer … ise” çıkarımdır.

Sonuç: $\neg A \to \neg B$.

Örnek 2. Yeni bir değişken eklenir: C: “Bu akşam ay tutulması var.”

Cümle: “Bu akşam uyanık olacağım, ama ay tutulması yoksa ay tutulmasını göremem.”

- Ay tutulması yoksa: $\neg C$, sonuç: $\neg B$.
- Uyanık olmak da şarttır, bu yüzden A ile ve bağlacıyla bağlanır.

Sonuç: $A \wedge \neg C \to \neg B$. Öncelik kuralına göre bu, $(A \wedge \neg C) \to \neg B$ demektir.

10.6 De Morgan kanunu ve denk ifadeler

Slayttaki kurallar (ekrandan, 01:36:30):

$$\neg(p \wedge q) \equiv \neg p \vee \neg q \qquad \neg(p \vee q) \equiv \neg p \wedge \neg q$$

$$p \to q \equiv \neg(p \wedge \neg q) \qquad \neg(p \to q) \equiv p \wedge \neg q \qquad p \to q \equiv \neg q \to \neg p$$

Bütün denklikler python ile dört değer kombinasyonunda doğrulanmıştır. Son denklik, $p \to q$’nun $\neg(p \wedge \neg q)$ biçiminden çıkar: değili $p \wedge \neg q$ olduğuna göre, aynı ifade $\neg q \to \neg p$ şeklinde de yazılabilir.

Slayttaki programlama örneği:

```
if (!(p() && q())) { /* ... */ }   // yerine,
if (!p() || !q())  { /* ... */ }   // daha hızlı olur.
```

Hocanın gerekçesi: ilk biçimde önce bir VE alınır, sonra sonucun değili alınır. İkincisinde değil işleminin maliyeti azdır, iki atomik bileşen vardır ve yalnız bunların VEYA’sına bakılır. Hoca bunun çok uç bir nokta olduğunu ve her zaman bu kadar ince ayrıntıya gerek olmadığını söyler. Asıl mesaj şudur: FOL ile kesin olarak tanımlanmış bir belirtim, bu dönüşümlerle daha sade ya da daha hızlı olacak şekilde yeniden modellenebilir. Ay tutulması örneğindeki $A \wedge \neg C \to \neg B$ için de cümle farklı yorumlanarak denk, daha kısa bir ifade bulunabilir.

> **Ders sonrası not:** Slayttaki “daha hızlı olur” iddiası kısa devre değerlendirmesi olan dillerde (C, Java vb.) geçerli görünmüyor. İki biçimde de önce p() çağrılır; p() yanlışsa q() hiç çağrılmaz ve sonuç doğrudur, p() doğruysa q() çağrılır. Yani aynı fonksiyonlar aynı sırada çalışır; fark büyük olasılıkla en fazla tek bir değil işlemidir ve derleyici bunu genellikle ortadan kaldırır. De Morgan’ın asıl yararı ifadeleri sadeleştirmek ve denk biçimler bulmaktır.

11. Birinci derece mantık (FOL)

11.1 Tanım

Slayttaki tanım (ekrandan, 01:40:00): birinci derece mantık, önerme mantığındaki bağlaçları canlandırarak nesnelerin özellikleri hakkında önermeler yapmamızı sağlar.

- İfadeler (predicates) ile nesnelerin özellikleri tanımlanır.
- Fonksiyonlar (functions) ile nesneler eşlenir.
- Niceleyiciler (quantifiers) ile çoklu nesneler için önerme kurulur.

11.2 İfadeler ve sabitler

Slayttaki örnek:

$$Sever(Sen, Yumurta) \wedge Sever(Sen, Domates) \to Sever(Sen, Menemen)$$

- Sen, yumurta, domates, menemen: sabitlerdir. Önerme değil, nesnedirler.

- Sever: ifadedir. Nesneleri argüman olarak alıp doğru/yanlış üretir. “Sen yumurtayı ya seversin ya sevmezsin.”
- $\wedge$, $\rightarrow$: bağlaçlardır.

Cümlenin anlamı: yumurtayı da domatesi de seviyorsan menemeni de seversin. Menemen ilk bakışta diğerlerinden bağımsız bir nesne gibi görünür, ama yumurta ve domatesten oluştuğu bilindiği için onu sevmeme durumu olamaz; bu FOL ile tanımlanmış olur.

11.3 Eşitlik

FOL'da özel bir ifade vardır: eşitlik ($=$). İki nesnenin birbirine eşit olduğunu gösterir ve yalnız nesnelere uygulanır, önermelere uygulanmaz. Önermeler için iki koşullu bağlaç ($\leftrightarrow$) kullanılır: ikisinin de aynı değerde olduğu durumda doğru olur.

11.4 Fonksiyonlar

Hocanın örneği, içten dışa okunur: “senin en sevdiğin filmin yıldızı”. FavoriteMovieOf(You) bir film nesnesi döndürür; doğru/yanlış değil nesne döndürdüğü için ifade değil fonksiyondur. StarOf(…) bu filmden bir aktör nesnesi üretir. “Senin en sevdiğin filmin yıldızı, onun en sevdiği filmin yıldızına eşittir” (ör. ikisi de Tom Hanks) cümlesinde iki taraf da nesne olduğu için eşittir kullanılabilir:

$$StarOf(FavoriteMovieOf(You)) = StarOf(FavoriteMovieOf(Her))$$

Diğer fonksiyon örnekleri: bir renk döndüren bir fonksiyon (hoca “yeşil döndürüyor herhalde” der), x, y, z'den hangisi ortanca değerse onu döndüren $Median(x, y, z)$ ve bir değer döndüren $x + y$. Hepsi nesne üretip geri döndürür.

11.5 Nesne ile ifadeyi karıştırmamak

> **Hocanın vurgusu:** Nesnelerle ifadeleri birbirine bağlamamaya ve ayrı tutmaya dikkat etmek gerekir.

- Nesnelere bağlaç uygulanamaz (eşitlik özel bir ifadedir). “Venus → TheSun” (“Venüs ise Güneş”) yanlıştır: ikisi de nesnedir.
- Fonksiyonlara önerme argüman olarak verilemez. Hocanın örneği: StarOf fonksiyonuna “IsRed(Sun) ∧ IsGreen(Mars)” gibi bir önerme verilirse, içeriden doğru döner ve “StarOf(doğru)” anlamsız olur.

Özetle: ifadeler doğru/yanlış döndürür ve bağlaçlarla birleştirilir; fonksiyonlar nesne döndürür ve bu nesneler uygun biçimde (ifadelerin argümanı olarak ya da eşitlikle) bağlanır.

12. Niceleyiciler ve kapsam

12.1 Evrensel ve varoluşsal niceleyici

Slayttaki tanımlar (ekrandan, 01:46:00):

Evrensel niceleyici (universal quantifier), $\forall$: “Tüm değişkenler için bu ifade doğrudur.” “All” anlamına gelir ve ters A ile yazılır. Uzaydaki bütün değişkenler için ifadenin mutlaka doğru olması gerekir; yazılımda bu şartın sağlatılması gerekir. Slayttaki örnek:

$$\forall n.(n \in \mathbb{N} \rightarrow (Even(n) \leftrightarrow Even(n^2)))$$

Slayttaki okuma: “For any natural number n, n is even if n² is even.” Her doğal sayı n için, n çift ise karesi de çifttir ve karesi çift ise n de çifttir; iki ifade ya ikisi birden doğru ya ikisi birden yanlıştır.

> **Ders sonrası not:** Hoca bu örneği anlatırken “tek sayının karesi tek olmayabiliyor mu? … kafam karıştı” diye duraksadı. Tek sayının karesi her zaman tektir: $(2k+1)^2 = 4k^2 + 4k + 1$. Bu yüzden iki koşullu ifade bütün doğal sayılar için doğrudur (python

ile 0–999 arasında da doğrulandı). Slayttaki İngilizce okuma (“n is even if n^2 is even”) formülün yalnız bir yönünü söyler; formül iki yönlüdür (“if and only if”).

Varoluşsal niceleyici (existential quantifier), $\exists$: “Bazı değişkenler için bu ifade doğrudur.” “There exists such an x that …” diye okunur: öyle bir şey vardır ki bu ifade onun için doğrudur. Tek bir örnek bile ifadeyi doğru yapar. Slayttaki örnekler:

$$\exists x.(Even(x) \wedge Prime(x))$$

Çift olan ve asal olan bir x vardır. Normalde bir çift sayının asal olması imkânsızdır, çünkü ikinin katıdır. Tek istisna 2’dir: hem çift hem asaldır. Bu yüzden ifade doğrudur. 2 asal olmasaydı $\exists$ kullanılamaz, “bütün x’ler için bu yanlıştır” diye bir ifade kurulması gerekirdi. (Python ile 2–49 arasında çift asal yalnız 2 çıktı.)

$$\exists x.(TallerThan(x, me) \wedge LighterThan(x, me))$$

Benden uzun ve benden hafif biri vardır. Böyle insanlar bulunabilir; bu yüzden bütün x’ler için değil, bazı x’ler için doğru olduğu $\exists$ ile ifade edilir.

12.2 Kapsam

Slayttaki kurallar (ekrandan, 01:50:00):

- Her niceleyicide tanıtılan değişken ve nicelenen ifade vardır.
- Değişken sadece nicelenen ifade kapsamında geçerlidir.

Örnek:

$$(\forall x.Loves(You, x)) \rightarrow (\forall y.Loves(y, You))$$

x yalnız soldaki parantezde, y yalnız sağdaki parantezde yaşar. Hoca sınıfa bunun Türkçe nasıl okunacağını sorar, cevap gelmeyince kendisi okur: “Siz herkesi severseniz, herkes de sizi sever.” Gerçek dünyada doğru olmayabilir, ama burada yazılım uzayında düşünülmektedir.

İkinci yazım:

$$(\forall x.Loves(You, x)) \rightarrow (\forall x.Loves(x, You))$$

Burada iki ayrı x vardır: soldaki parantezde bir x örneği, sağdakinde başka bir x örneği yaşar (x1 ve x2 gibi). Bu yazım da doğrudur ve birebir aynı şeyi söyler, ama okunaklılık azalır ve kafa karıştırabilir. Hoca kendisinin genellikle ilk biçimi kullandığını ve onun tercih edilmesini önerir.

12.3 Yanlış parantezleme

Hocanın örneği: “her x için, P(x) veya R(x) ise Q(x)” söylenmek isteniyor. Şöyle yazılırsa:

$$(\forall x.P(x)) \vee R(x) \rightarrow Q(x)$$

x yalnız ilk parantezde tanımlıdır; R(x) ve Q(x)’teki x tanımsız kalır. Bu özellikle yanlış yapılmış bir parantezlemedir. Doğrusu bütün ifadeyi tek bir kapsamda almaktır:

$$\forall x.(P(x) \vee R(x) \rightarrow Q(x))$$

Hocanın vurgusu: “Her x” deniyorsa bu bütün ifadeyi kaplamalıdır; bütün ifade boyunca geçerli bir x değişkeni olmalıdır.

13. İfadeleri kurgulamak

13.1 "Bütün P'ler Q'dur"

Slayttaki örnek (ekrandan, 01:54:40): "All puppies are cute" (bütün köpek yavruları sevimlidir).

İlk deneme:

$$\forall x.(Puppy(x) \land Cute(x))$$

İngilizcesi doğru gibi okunur, ama ifade yanlıştır. Slayttaki açıklama: elimizde puppy dışındaki x'ler için de doğru olması gerekir; İngilizcesi doğru olsa bile FOL ifadesi yanlıştır. Hocanın sözleriyle bu ifade, puppy dışındaki x'ler için de doğru olmasını gerektirir.

Doğrusu çıkarımla yazmaktır:

$$\forall x.(Puppy(x) \rightarrow Cute(x))$$

"Eğer x puppy ise cute'tur." Genel kalıp:

$$\text{All P's are Q's:} \quad \forall x.(P(x) \rightarrow Q(x))$$

Hoca bunu ezber gibi düşünülebilecek standart bir kalıp olarak verir.

> **Ek bilgi (derste anlatılmadı):** $\forall x.(Puppy(x) \land Cute(x))$, uzaydaki her nesnenin hem köpek yavrusu hem sevimli olduğunu söyler; uzayda tek bir kedi bile varsa yanlış olur. $\forall x.(Puppy(x) \rightarrow Cute(x))$ ise köpek yavrusu olmayan nesneler için kendiliğinden doğrudur (çıkarımın sol tarafı yanlışsa sonuç doğrudur, doğruluk tablosunun son iki satırı) ve yalnız köpek yavrularının sevimli olmasını şart koşar.

Bu noktada bir öğrenci soru sorar (aşağıda Örnek Sorular, Soru 3).

13.2 "Bazı P'ler Q'dur"

Slayttaki örnek: "Some blobfish is cute" (bazı damla balıkları sevimlidir). Hoca damla balığının (blobfish) ne olduğunu sorar; sınıfta "kirpi balığı, balon balığı" gibi tahminler gelir, sonunda internette bakılıp Türkçesinin "damla balığı" olduğu bulunur. Hocaya göre dünyanın en çirkin balığı seçilmiştir; örnek için sevimli olma olasılığının olmadığı varsayılır.

İlk deneme:

$$\exists x.(Blobfish(x) \rightarrow Cute(x))$$

Slayttaki açıklama: en azından bir örnek için doğru olmalıdır; İngilizcesi doğru olsa bile FOL ifadesi yanlıştır. Hoca, sevimli bir damla balığı hiç olmasa da bu ifadenin ("damla balığıysa sevimlidir" diye bir örnek vardır) istenen anlamı vermediğini söyler.

Doğrusu:

$$\exists x.(Blobfish(x) \land Cute(x))$$

Genel kalıp:

$$\text{Some P is a Q:} \quad \exists x.(P(x) \land Q(x))$$

> **Ek bilgi (derste anlatılmadı):** $\exists x.(Blobfish(x) \rightarrow Cute(x))$ çok zayıf bir ifadedir: uzayda damla balığı olmayan tek bir nesne bulunması onu doğru yapar, çünkü o nesne için çıkarımın sol tarafı yanlıştır. Bu yüzden "bazı damla balıkları sevimlidir" anlamını vermez; varoluşsal niceleyici $\land$ ile, evrensel niceleyici $\rightarrow$ ile kullanılır.

Hocanın vurgusu: Bu iki kalıp standarttır ve temeldir; karmaşık ifadeler bunlar kullanılarak kurgulanır.

13.3 "Herkes başka birini sever"

Slayttaki ifade, renklerle cümlenin parçalarına eşlenmiştir (ekrandan, 02:00:00). Tanımlar: $Person(p)$ ve $Loves(x, y)$, ikisi de doğru/yanlış döndüren ifadelerdir.

"Everybody loves someone else":

$$\forall p.(Person(p) \rightarrow \exists q.(Person(q) \land p \neq q \land Loves(p, q)))$$

"For every person there is some person who isn't them that they love."

Kuruluşu adım adım:

1. "Everybody": $\forall p.(Person(p) \rightarrow ...)$. Bütün p'ler için, $Person(p)$ doğruysa, yani "her insan için" demektir.
2. "Someone": öyle biri vardır; varoluşsal niceleyici gerekir: $\exists q$. q'nun da insan olması için $Person(q)$ eklenir.
3. "Else": kendini değil başka birini sevmelidir: $p \neq q$.
4. "Loves": $Loves(p, q)$ doğru olmalıdır.
5. Üç koşul $\land$ ile bağlanır: q insandır, p'ye eşit değildir ve p, q'yu sever. Böyle bir q'nun varlığı $\rightarrow$ ile "p insandır" koşuluna bağlanır ve bu bütün p'ler için geçerlidir.

Hoca bu sayede basit bir İngilizce cümlenin, kim okursa aynı şeyi anlayacağı değişmez bir mantık ifadesine dönüştüğünü vurgular.

13.4 "Herkesin sevdiği biri vardır"

"There is a person that everyone else loves" (öyle bir insan vardır ki kendisi dışındaki herkes onu sever):

$$\exists p.(Person(p) \land \forall q.(Person(q) \land p \neq q \rightarrow Loves(q, p)))$$

"There is some person who everyone who isn't them loves."

Kuruluşu:

1. "There is a person": $\exists p.(Person(p) \land ...)$, bütün ifadeyi kaplayacak şekilde. Varoluşsal niceleyici $\land$ ile kullanılır.
2. "Everyone else": kendisi olmayan herkes; evrensel niceleyici gerekir: $\forall q$.
3. Kalan bütün insanlar: $Person(q) \land p \neq q$; ikisi birden doğru olmalıdır.
4. Bu durumda q, p'yi sevmelidir: $\rightarrow Loves(q, p)$. Evrensel niceleyici $\rightarrow$ ile kullanılır.

14. Niceleyici sırası ve değilleme

14.1 Niceleyici sıralama

Hem $\forall$ hem $\exists$ varsa sıra anlamı değiştirir. Slayttaki ifadeler (ekrandan, 02:05:00):

$$\forall x.\exists y.P(x, y)$$

"For any choice x, there's some y where P(x, y) is true." Her x seçimi için P(x, y)'yi doğru yapan bir y bulunabilir.

$$\exists x.\forall y.P(x, y)$$

"There is some x where for any choice of y, we get that P(x, y) is true." Öyle bir x vardır ki y ne seçilirse seçilsin P(x, y) doğrudur.

Ders sonrası not: Hoca ikinci ifadeyi sözlü olarak "y'nin bütün seçenekleri için P(x, y)'yi doğru yapan bazı x'ler bulabiliriz" diye okudu. Bu okuma aslında $\forall y.\exists x.P(x,y)$'nin anlamıdır (her y için x ayrı ayrı seçilebilir). Slayttaki doğru okumada tek bir x vardır ve bu aynı x bütün y'ler için çalışır.

Ek bilgi (derste anlatılmadı): $\exists x.\forall y.P(x,y)$ doğruysa $\forall y.\exists x.P(x,y)$ de doğrudur (hep o tek x seçilir); tersi genelde doğru değildir. Örneğin $\forall x.\exists y.Loves(x,y)$ ("herkes birini sever"; herkesin sevdiği kişi farklı olabilir) ile $\exists y.\forall x.Loves(x,y)$ ("herkes tarafından sevilen tek bir kişi vardır") farklı iddialardır.

14.2 Niceleyicileri değillemek

Hocanın kuralı: değil işaretini niceleyicinin içine itin ve niceleyiciyi diğeriyle değiştirin; $\forall$ ise $\exists$'e, $\exists$ ise $\forall$'a çevirin.

$$\neg\forall x.P(x) \equiv \exists x.\neg P(x) \qquad \neg\exists x.P(x) \equiv \forall x.\neg P(x)$$

Hocanın açıklaması: "tüm x'ler için P(x)" ifadesi ne zaman yanlış olur? $\neg P(x)$'i doğru yapan x'ler varsa. Tersinden de aynı kural işler: $\neg\exists x.\neg P(x)$'te değil içeri itilince iki değil birbirini götürür ve $\forall x.P(x)$'e geri dönülür.

14.3 Örnek: "Herkes birini sever" cümlesinin değili

Slayttaki adımlar (ekrandan, 02:10:00):

$\forall x.\exists y.Loves(x,y)$ "Everyone loves someone."

1. Başa değil konur: $\neg\forall x.\exists y.Loves(x,y)$
2. Değil içeri itilir, $\forall$ $\exists$'e döner: $\exists x.\neg\exists y.Loves(x,y)$
3. Değil yine içeri itilir, $\exists$ $\forall$'a döner: $\exists x.\forall y.\neg Loves(x,y)$

Sonuç: "There's someone who doesn't love anyone." Öyle bir x vardır ki bütün y'leri sevmez, yani kimseyi sevmeyen biri vardır. Hoca okurken önce karıştırır ("bazıları vardır ki kimse onu sevmez" der); bir öğrencinin sorusu üzerine doğru okumada uzlaşılır (Örnek Sorular, Soru 4). Hoca bu kişiyi şakayla "taş kalpli" diye niteler.

Hoca bu konunun kafa karıştırıcı olduğunu kabul eder, ama değişmezleri düzgün tanımlayabilmek için bunlara dikkat etmek gerektiğini söyler.

15. Küme işlemleri ve OCL'ye geçiş

15.1 Temel tanımlar

Slayttaki tanımlar (ekrandan, 02:11:30):

- $Set(s)$: s bir kümedir.
- $x \in y$: x, y'nin bir elemanıdır.

15.2 Boş küme vardır

"Empty set exists" cümlesini yazmak için: öyle bir S vardır ki S bir kümedir ve S'nin elemanı olan hiçbir x yoktur.

$$\exists S.(Set(S) \land \neg\exists x.x \in S)$$

Değil içeri itilip $\exists$ $\forall$'a çevrilince:

$$\exists S.(Set(S) \land \forall x.\neg(x \in S))$$

Bütün x'ler S kümesinin dışındadır: elemanı olmayan ama küme olan bir küme vardır. Bu, tanım olarak bildiğimiz boş kümedir. İki ifade aynı şeyi söyler; boş küme ikisinden biriyle biçimsel olarak ifade edilebilir.

15.3 Küme eşitliği

"Two sets are equal if and only if they contain the same elements":

$$\forall S.(Set(S) \rightarrow \forall T.(Set(T) \rightarrow (S = T \leftrightarrow \forall x.(x \in S \leftrightarrow x \in T))))$$

Okunuşu:

1. Bütün S'ler için, S bir kümeyse;
2. bütün T'ler için, T de bir kümeyse;
3. $S = T$ ancak ve ancak bütün x'ler için "x, S'nin elemanıdır" ile "x, T'nin elemanıdır" eşdeğerse.

S ile T eşitse içlerindeki elemanların hepsi de eşit olmalıdır; elemanlar birebir aynıysa kümeler de eşittir. Biri doğruysa diğeri doğru, biri yanlışsa diğeri yanlıştır. $Set(S) \rightarrow$ ve $Set(T) \rightarrow$ parçaları, bunlardan biri küme değilse ifadenin konuşulmasına gerek olmadığını gösterir. Neden $\forall T$ kullanıldığı derste soru-cevapla tartışılır (Örnek Sorular, Soru 5).

15.4 Kümeyle sınırlı niceleyiciler

Slayttaki ifadeler (ekrandan, 02:14:40):

$$\forall x \in S.P(x) \qquad \text{"for any element x of set S, P(x) holds."}$$

$$\exists x \in S.P(x) \qquad \text{"there is an element x of set S where P(x) holds."}$$

Burada şart uzaydaki bütün x'ler için değil, yalnız bir kümenin içindeki x'ler için konmuştur; hocanın deyişiyle uzay bir kümeye ufaltılmıştır. Bu, uygulamada bölümlemeler (partition) yapmayı sağlar: bazı değişkenler bir kümede, bazıları başka bir kümede olur; bir küme için doğru olan şey başka bir küme için yanlış olabilir ya da farklı ifade edilebilir. Hocanın kütüphane örneğine bağlantısı: 12 yaşından küçük çocuklar için 5 kitap sınırı vardır, büyükler için yoktur; bu kısıt FOL ile yazılmak istenirse kümelenerek ifade edilebilir.

15.5 OCL'ye geçiş

Sıradaki konu OCL'dir. OCL, şimdiye kadar görülen matematiksel ifadeleme dilinin, yani FOL'un alternatifidir. Bir sözdizimi verir ve bu sözdizimi doğrudan FOL'a dönüştürülebilir. Sözde kod yazar gibi yazıldığı için okunaklılığı daha yüksektir; ayrıca UML diyagramlarına açıklama ya da kısıt eklenebilir. Hoca örnek olarak 12 yaşından küçük çocukların 5 kitap sınırının OCL ile rahatlıkla ifade edilebileceğini söyler. OCL'nin tamamı, yarım kalmasın diye sonraki derse bırakılır.

Örnek Sorular

Soru 1 (ekrandan, 00:40:00). Slayttaki 11 maddelik kütüphane gereksinimlerinden (Bölüm 5) bir kavramsal sınıf modeli çıkarın. Hangi aday sınıflar elenmeli, hangi ilişkiler ve ilişki sınıfları kurulmalıdır?

Çözüm (derste verildi):

- Aday sınıflar (ekrandan, 00:44:10): KütüphaneKartı, Ziyaretçi, Ceza, Parça, Çocuk, BaşvuruKitabı, Dergi, Kitap, DVD, ÇokSatan, Para, Tarih.
- Eleme: KütüphaneKartı yalnız numara tutar ve ziyaretçiyle birebirdir, Ziyaretçi'nin `KKartNo` özelliği olur. Çocuk yalnız yaşıyla ayrılır, Ziyaretçi'ye yaş özelliği eklenir. ÇokSatan, Kitap'ın `çokSatan: Boolean` özelliği olur. Değişebilen durum alt sınıf yapılmaz.

- Genelleştirme: Parça; ÖdünçVerilemezParça (BaşvuruKitabı, Dergi) ve ÖdünçVerilebilirParça (Kitap, DVD). Ödünç alma yalnız ÖdünçVerilebilirParça'ya bağlanır.
- Talep, kopyaya değil Başlık sınıfına bağlanır (Başlık 1, Parça * birleşme; Başlık * – * Ziyaretçi `talepEder`).
- Teslim tarihi, uzatma durumu, ceza ve getirme tarihi Ödünç ilişki sınıfına taşınır; fiyat parçada kalır.
- Son model Bölüm 8'deki gibidir (ekrandan, 01:00:00).

Soru 2 (derste sözlü, 01:50:30). $(\forall x.Loves(You, x)) \rightarrow (\forall y.Loves(y, You))$ ifadesini anlamlı bir Türkçe cümle olarak okuyun.

> **Çözüm (derste verildi):** Sınıftan cevap gelmeyince hoca okudu: "Siz herkesi severseniz, herkes de sizi sever." x yalnız soldaki, y yalnız sağdaki parantezde tanımlıdır. Sağdaki değişkene de x adı verilebilir; ifade yine doğrudur ama okunaklılık azalır.

Soru 3 (derste sözlü, 01:56:25). Bir öğrenci sordu: "Bütün köpek yavruları sevimlidir" örneğinde, alttaki ifade ($\forall x.(Puppy(x) \rightarrow Cute(x))$) x köpek yavrusu olmadığında da doğru dönmüyor mu? Üstteki ($\forall x.(Puppy(x) \wedge Cute(x))$) neden yanlış?

> **Çözüm (derste verildi):** Hoca şöyle cevapladı: söylenmek istenen, bütün köpek yavrularının sevimli olduğu, ama bunların dışında da sevimli şeyler (ör. kedi yavruları) bulunduğudur; bütün sevimliler köpek yavrularından oluşmaz. Öğrenci de yanlışlığın buradan geldiğini söyledi.
>
> **Ders sonrası not:** Öğrencinin gözlemi doğrudur: çıkarım biçimi, köpek yavrusu olmayan x'ler için kendiliğinden doğrudur ve istenen de budur. $\wedge$ biçiminin yanlış olmasının nedeni, uzaydaki her nesnenin hem köpek yavrusu hem sevimli olduğunu iddia etmesidir; uzayda bir kedi bile varsa yanlış olur. Hocanın cevabı (sevimli olan başka şeyler de vardır) bu durumun bir örneğidir: kedi yavrusu sevimlidir ama köpek yavrusu değildir, bu yüzden $Puppy \wedge Cute$ onun için yanlıştır.

Soru 4 (ekrandan, 02:10:00; derste sözlü, 02:09:48). $\forall x.\exists y.Loves(x, y)$ ("Everyone loves someone") ifadesini değilleyin ve sonucu okuyun. Sonuç "bazıları kimseyi sevmez" mi, yoksa "bazılarını kimse sevmez" mi demektir?

> **Çözüm (derste verildi):**
>
> - $\neg\forall x.\exists y.Loves(x, y)$
> - $\equiv \exists x.\neg\exists y.Loves(x, y)$
> - $\equiv \exists x.\forall y.\neg Loves(x, y)$
>
> "There's someone who doesn't love anyone": kimseyi sevmeyen biri vardır. Bir öğrenci hocanın okumasını sorgulayınca hoca düzeltti: x birinci argümandır, yani seven taraftır; $\neg Loves(x, y)$ "x, y'yi sevmez" demektir. Bütün y'ler için doğru olduğuna göre x kimseyi sevmez. "Kimsenin sevmediği biri vardır" okuması yanlıştır.

Soru 5 (derste sözlü, 02:13:20). Küme eşitliği ifadesinde, $\forall S.(Set(S) \rightarrow \forall T.(Set(T) \rightarrow (S = T \leftrightarrow \forall x.(x \in S \leftrightarrow x \in T))))$, neden "her T için" dendi, "bazı T'ler için" denmedi?

> **Çözüm (derste verildi):** Sınıftaki cevap ve hocanın onayı: eşitlik tanımı bütün S ve T kümeleri için doğru olmalıdır. Şartı sağlamayan bir T bulunabiliyorsa eşitlik tanımlanmış olmaz. Amaç, bir uygulamada karşılaşılabilecek bütün olasılıkları ele alan kesin bir tanım yapmaktır. $Set(S) \rightarrow$ ve $Set(T) \rightarrow$ koşulları da bunlardan biri küme değilse ifadenin konu dışı kalmasını sağlar.

Dersin Toparlaması

Dersin ilk bölümünde tasarım, analizden sonra gelen ve belirli bir mimariye uygun yazılım uzayı modelini oluşturan aşama olarak tanımlandı. Tasarım kararlar vermektir ve bu kararlar çoğunlukla işlevsel olmayan gereksinimler arasında ödün vermekle ilgilidir. Karar vermek zorlaştığında simülasyonlar, prototipler ve bina mimarisinden miras alınan tasarım çalışmaları kullanılır; bir tasarım çalışmasında en az iki çözüm denenir, kriterler belirlenip ölçülür ve sonuçlar dokuz bölümlü bir raporla sunulur. Kütüphane örneği, gereksinimlerden kavramsal modele giden yolu yeniden gösterdi: önce bütün adaylar ortaya konur, sonra değişebilen

durumlar özelliğe dönüştürülerek elenir, genelleştirmeyle davranışlar modelin yapısına gömülür, talep başlığa bağlanır ve ilişkiye ait özellikler bir ilişki sınıfına taşınır. Sonraki adım bu kavramların servislere, veri depolarına ve kuyruklara dönüşeceği tasarım kararlarıdır.

İkinci bölümde biçimsel belirtime geçildi. Önerme mantığının bağlaçları, doğruluk tablosu, operatör öncelikleri ve De Morgan kuralları hatırlatıldı. Birinci derece mantıkta ifadeler, fonksiyonlar, eşitlik ve niceleyiciler tanıtıldı; “bütün P’ler Q’dur” ($\forall$ ile $\rightarrow$) ve “bazı P’ler Q’dur” ($\exists$ ile $\wedge$) kalıpları, kapsam ve parantezleme kuralları, niceleyici sırası ve değilleme örneklerle gösterildi. Ders, boş kümenin ve küme eşitliğinin FOL ile tanımlanması ve niceleyicilerin kümelerle sınırlanmasıyla bitti. Hoca bir sonraki derste aynı belirtimlerin daha okunaklı karşılığı olan OCL’ye ve onun UML diyagramlarıyla kullanımına geçileceğini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.